Three.js - A 3D Scene Graph API

Three.js

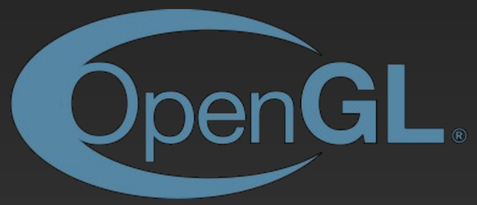
open source JavaScript API
for creating and displaying
animated 3D computer graphics
on a web browser.


carvisualizer.plus360degrees.com/threejs/
SELECT CAR MODEL


Chill the lion
REFRESH THE LION FROM THE BLISTERING HEAT
GO

Three.js is a 3D library that tries to make it as easy as possible to get 3D content on a webpage.

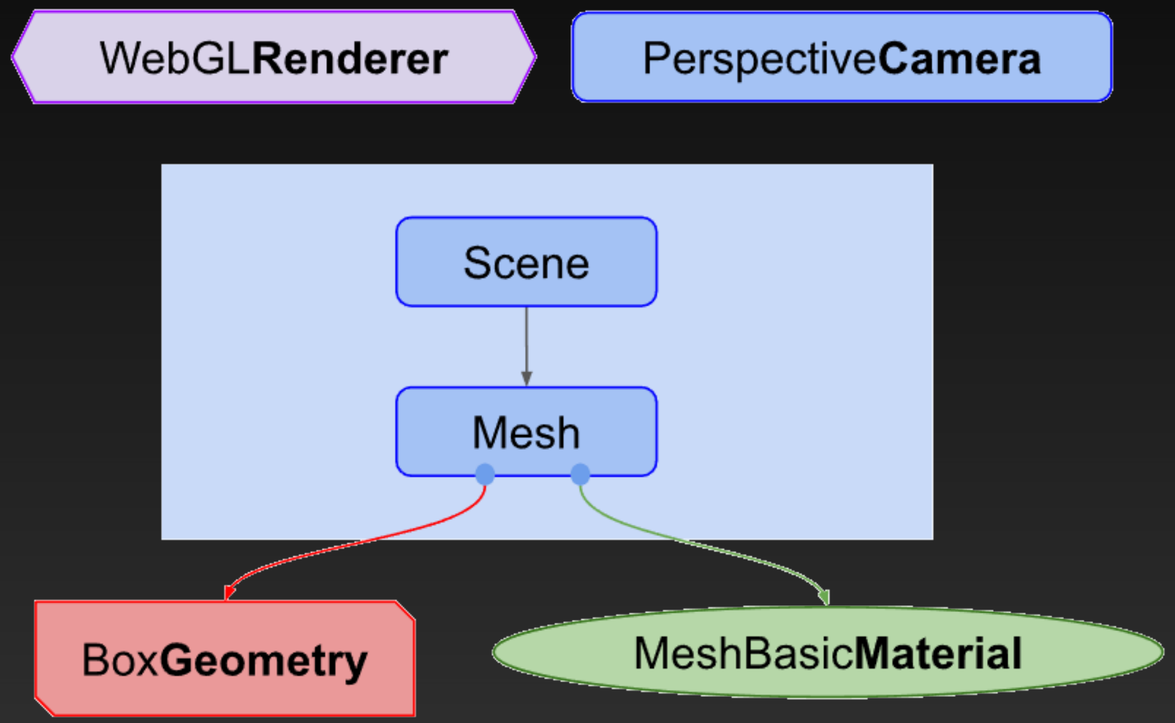
WebGL generally requires quite a bit of code and that is where three.js comes in.







Hello Cube

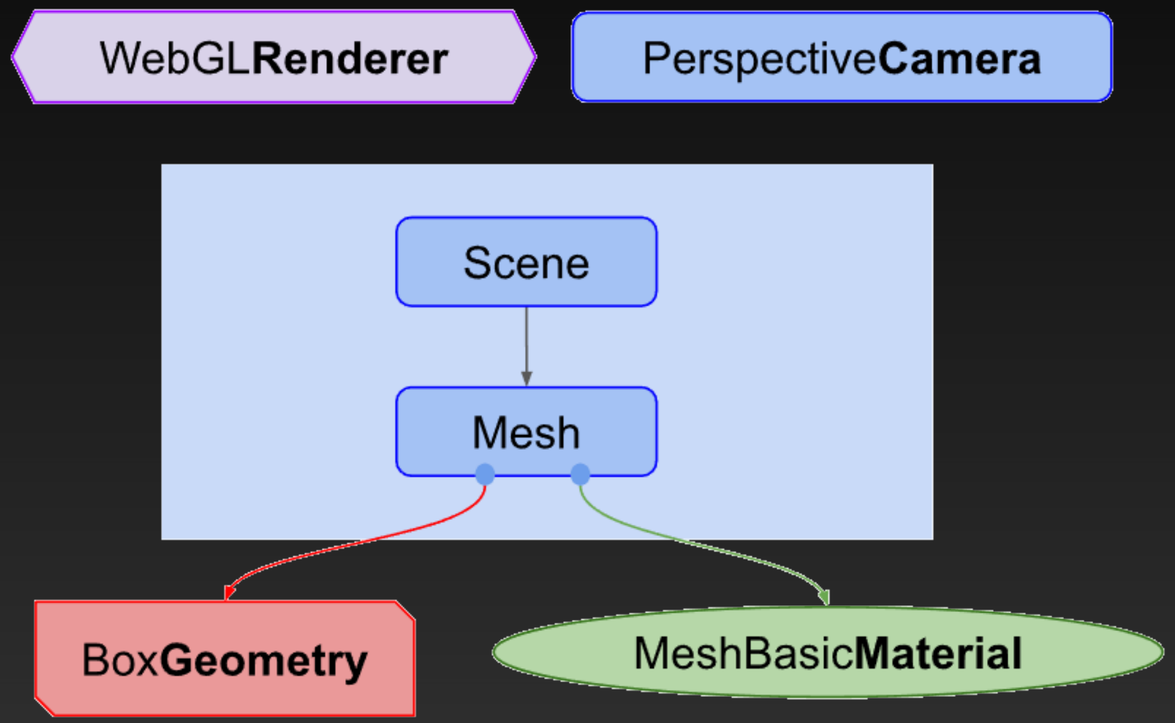


Renderer

You pass a Scene and a Camera to a Renderer

It renders (draws) the portion of the 3D scene that is inside the frustum of the camera as a 2D image to a canvas

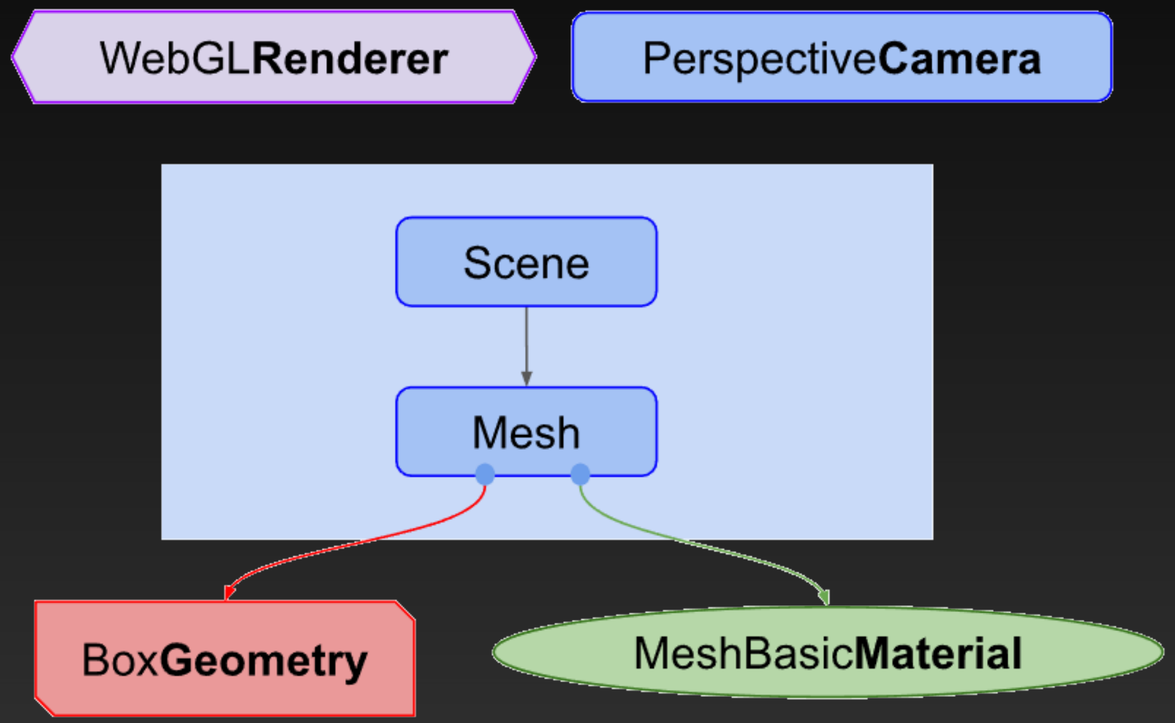
Hello Cube



Scene
Scene object defines the root of the scenegraph.

Mesh
Mesh objects represent drawing a specific Geometry with a specific Material.

Hello Cube



Geometry
Geometry objects represent the vertex data of some piece of geometry like a sphere, cube, plane, tree, building, etc...

Material
Material objects represent the surface properties used to draw geometry including things like the color to use and how shiny it is.

Skelton

```
<!DOCTYPE html>
<html>
<head>
    <title>My first three.js app</title>
    <style>
        body { margin: 0; }
    </style>
</head>

<body>
    <canvas id="c"></canvas>
    <script type="module">
        import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
        import { OrbitControls } from "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

        const renderer;
        const scene;
        const camera;
    </script>
</body>
</html>
```

無邊界， body 會佔滿整個視窗

var	函數
let	區塊
const	區塊

skeleton

```
<script type="module">
import * as THREE from …;
</script>
```

type="module" enables us to use the import keyword to load three.js

skeleton

```
import * as THREE from
"https://cdn.skypack.dev/three@0.132.2";
```

import everything from the three.js core under the THREE namespace

```
 import { OrbitControls } from
"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";
```

importing plugins (a camera controls plugin)

Hello cube - draw into that canvas

```
const canvas = document.
getElementById('c');

const renderer = new
THREE.WebGLRenderer({antialias
: true, canvas});
...
```



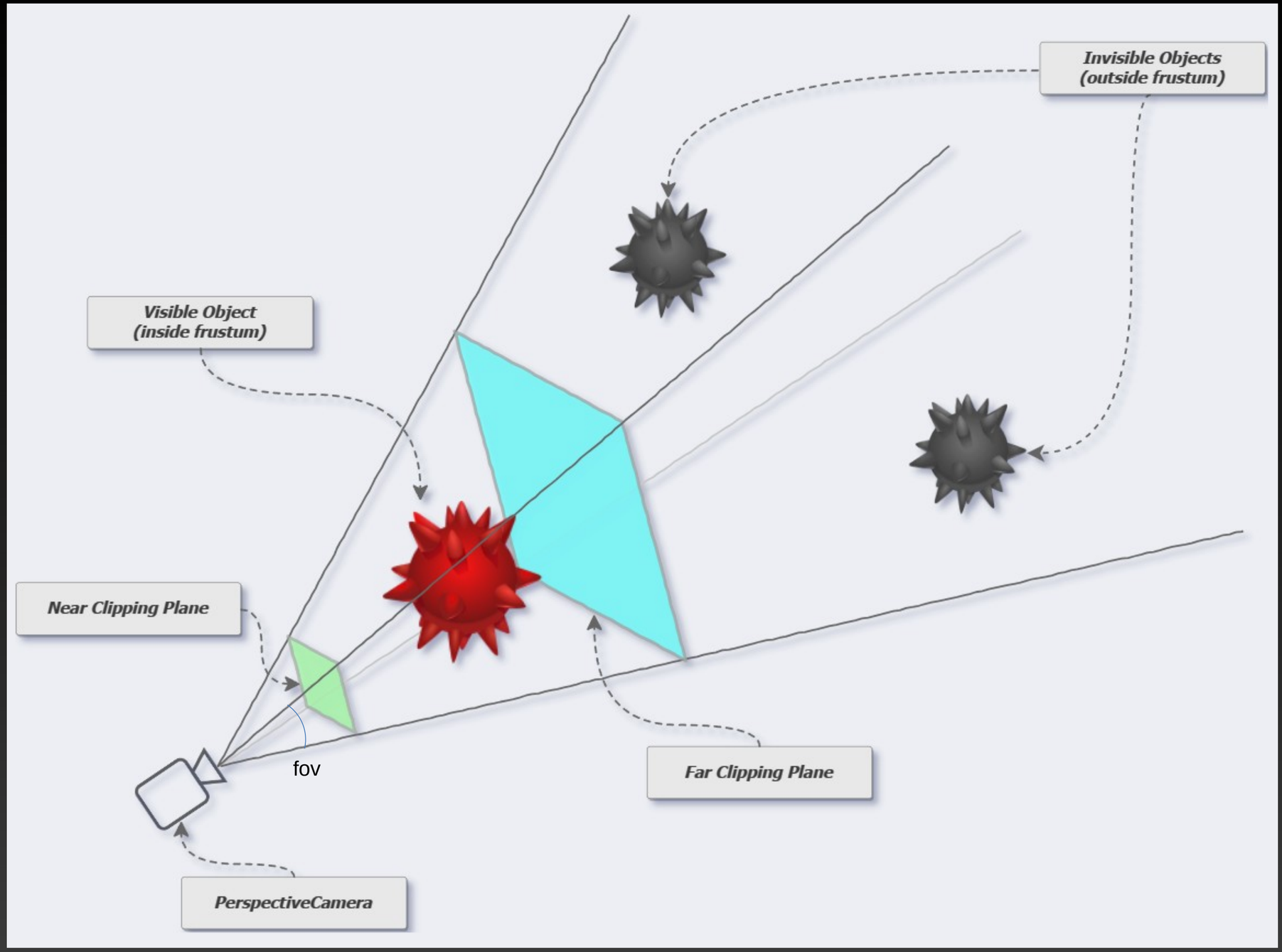
WebGLRenderer ： The renderer is the thing responsible for actually taking all the data you provide and rendering it to the canvas

Hello cube - create a camera

```
const fov = 75;
const aspect = 2;  // the canvas default
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
```

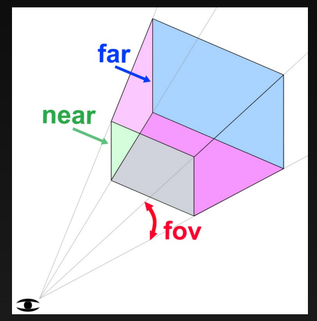
fov is short for field of view. In this case 75 degrees in the vertical dimension.

aspect is the display aspect of the canvas. By default a canvas is 300x150 pixels which makes the aspect 300/150 or 2.

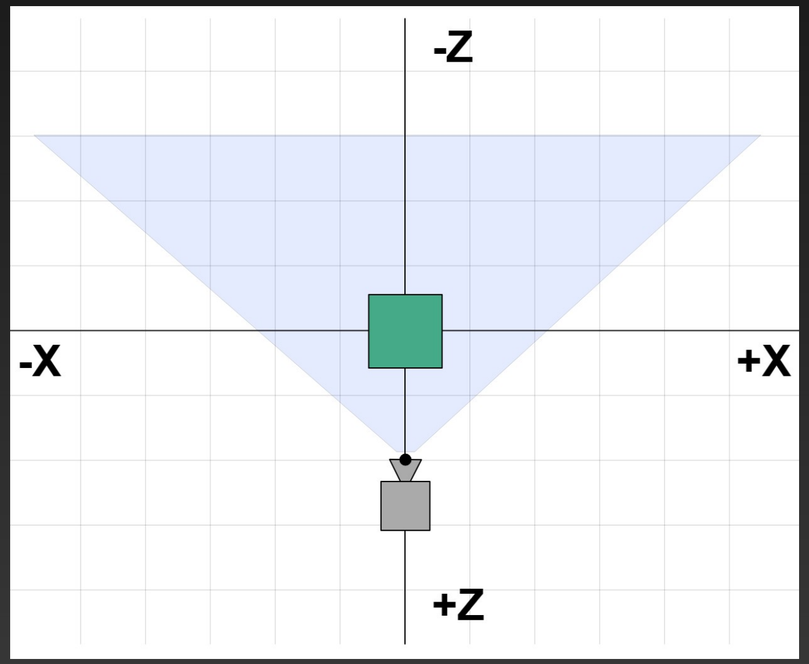

Invisible Objects
(outside frustum)
Visible Object
(inside frustum)
Near Clipping Plane
fov
Far Clipping Plane
PerspectiveCamera

Hello cube - create a camera

camera.position.z = 2;



Those settings define a "frustum". Anything inside the defined frustum will be drawn.



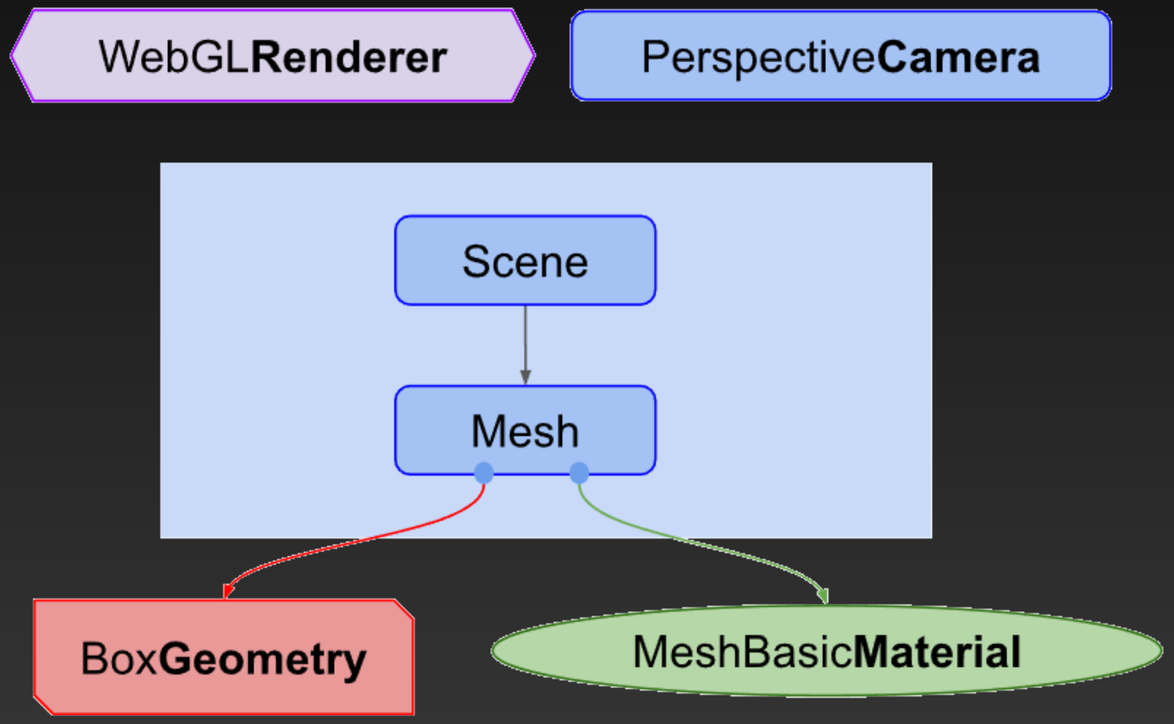
camera defaults to looking down the -Z axis with +Y up.

put a cube at the origin so we need to move the camera back a little

Hello cube - make a Scene

const scene = new THREE.Scene();

Anything you want three.js to draw needs to be added to the scene.

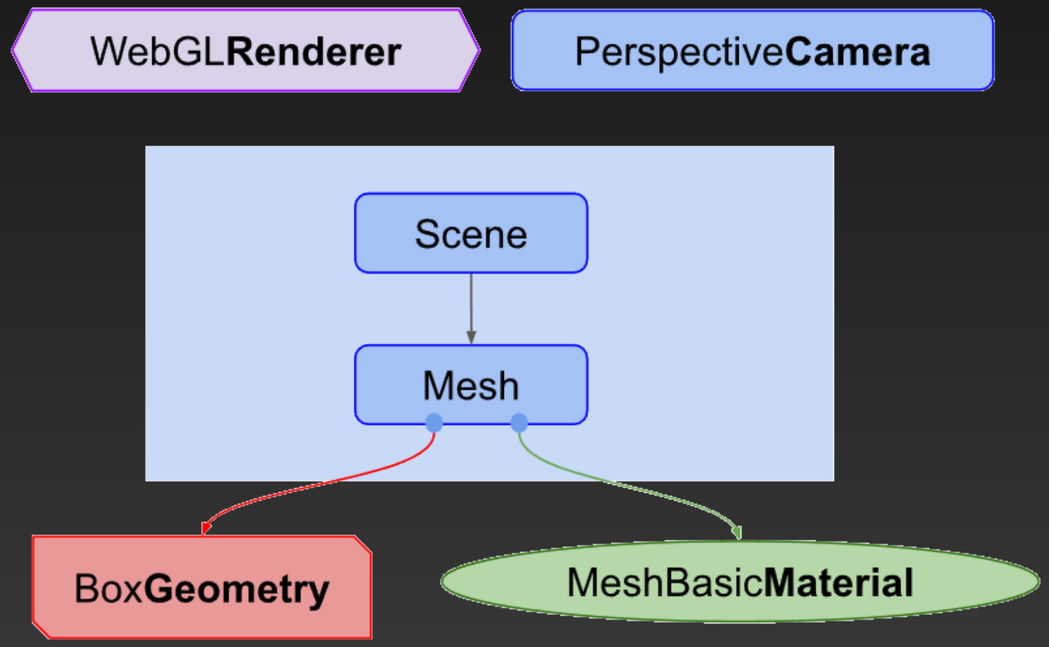


Hello cube - create a BoxGeometry and basic material

```
const boxWidth = 1;
const boxHeight = 1;
const boxDepth = 1;
const geometry = new
THREE.BoxGeometry(boxWidth,
boxHeight, boxDepth);

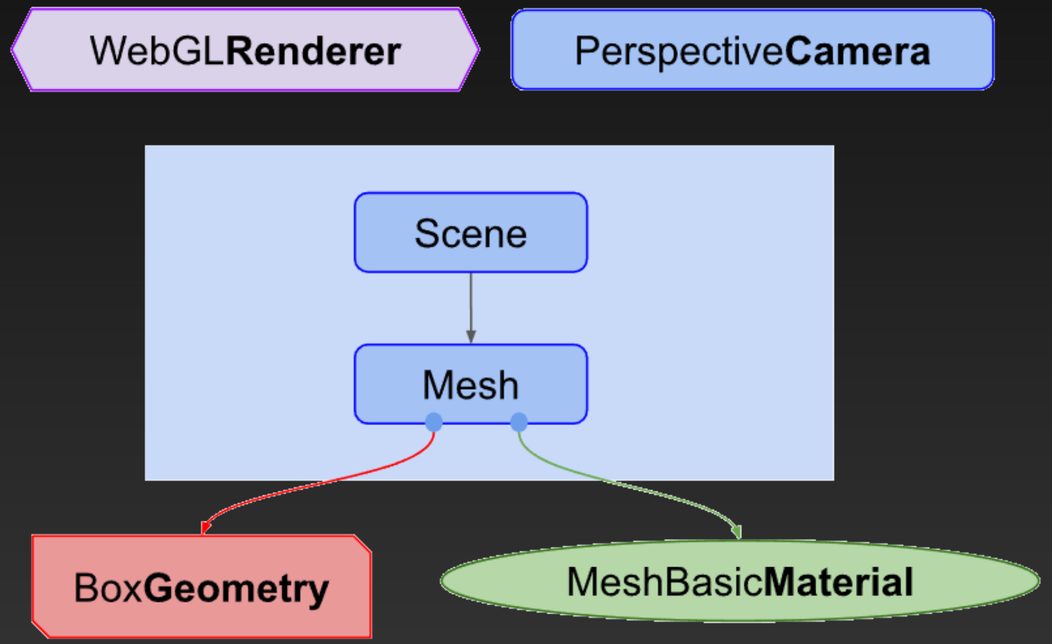
const material = new
THREE.MeshBasicMaterial({color:
0x00FF00});
```

BoxGeometry contains the data for a box which defines the vertices that make up our 3D object. Then create a basic material and set its color(green).



Hello cube - create a mesh

const mesh = new THREE.Mesh(geometry, material);



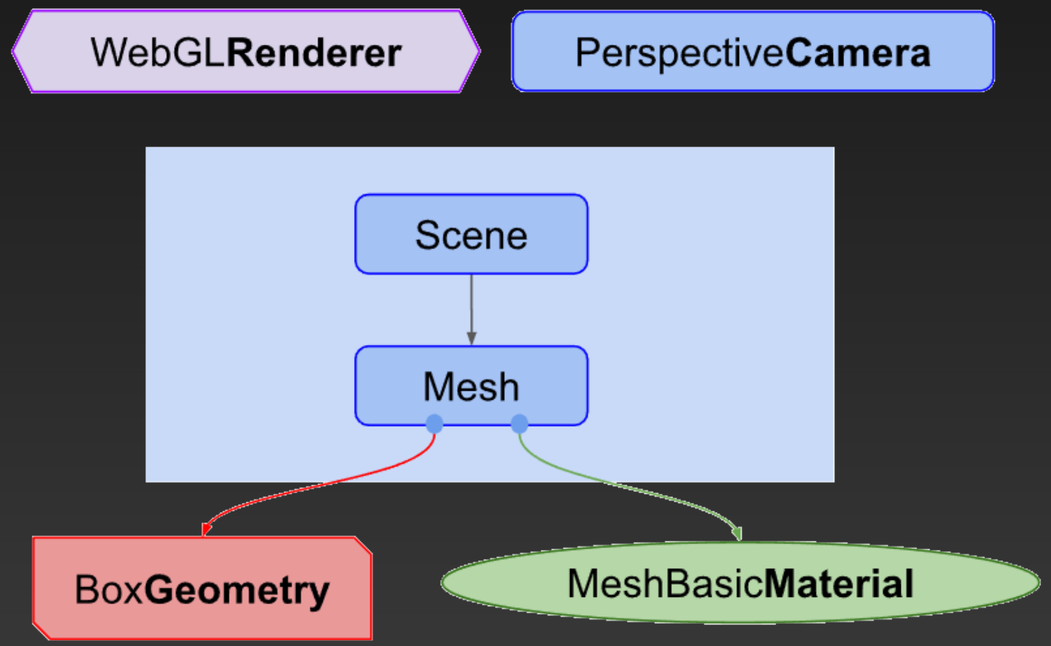
Mesh
1. A Geometry (the shape of the object)
2. A Material (how to draw the object, shiny or flat, what color, what texture(s) to apply. Etc.)
3. The position, orientation, and scale of that object

Hello cube - add mesh to scene and render

scene.add(mesh);

renderer.render(scene, camera);

render the scene by calling the renderer's render function



Hello cube - animate the cube

```
function render(time) {
  time *= 0.001;  // convert time to
seconds

  mesh.rotation.x = time;
  mesh.rotation.y = time;

  renderer.render(scene, camera);

  requestAnimationFrame(render);
}
requestAnimationFrame(render);
```

requestAnimationFrame(callback)

requestAnimationFrame() method tells the browser that you wish to perform an animation.

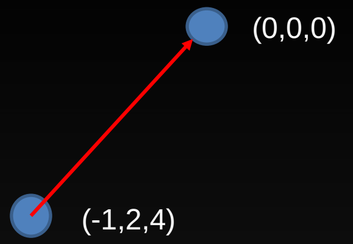
callback function
A function to call when it is time to update your animation. The callback has one single argument, a DOMHighResTimeStamp, which indicates the current time.

Hello cube - output in javascript

```
…
<p id="demo"></p>
…
document.getElementById("demo")
.innerText = (camera.position.z);
…
```

Use the innerText property to change the inner text of the HTML element

Hello cube – add a light




```
const color = 0xFFFFFF;
const intensity = 3;
const light = new
THREE.DirectionalLight(color,
intensity);
light.position.set(-1, 2, 4);
scene.add(light);
```

Directional lights have a position and a target. Both default to 0, 0, 0.

Change material to a MeshPhongMaterial which is affected by lights.

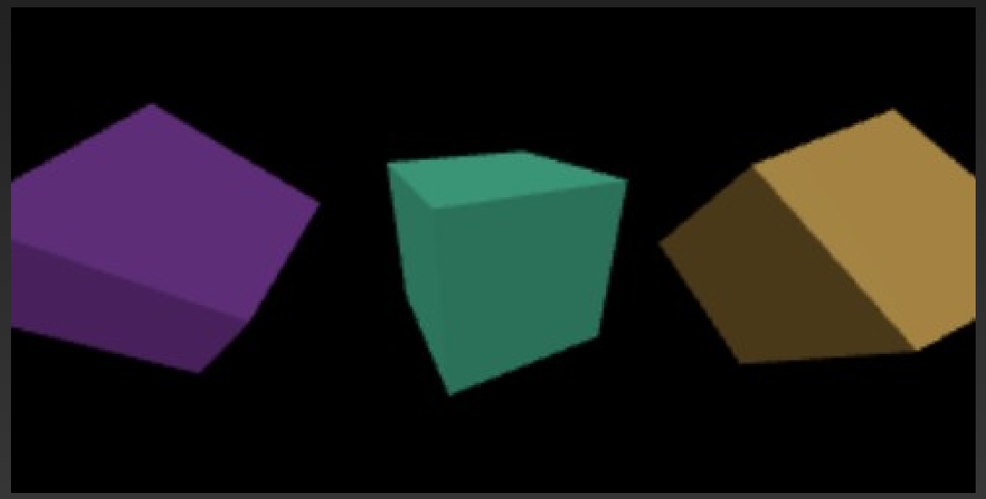
```
- const material = new THREE.MeshBasicMaterial({color: 0x22FF33});
+ const material = new THREE.MeshPhongMaterial({color: 0x22FF33});
```

Lab

Add 2 more cubes:

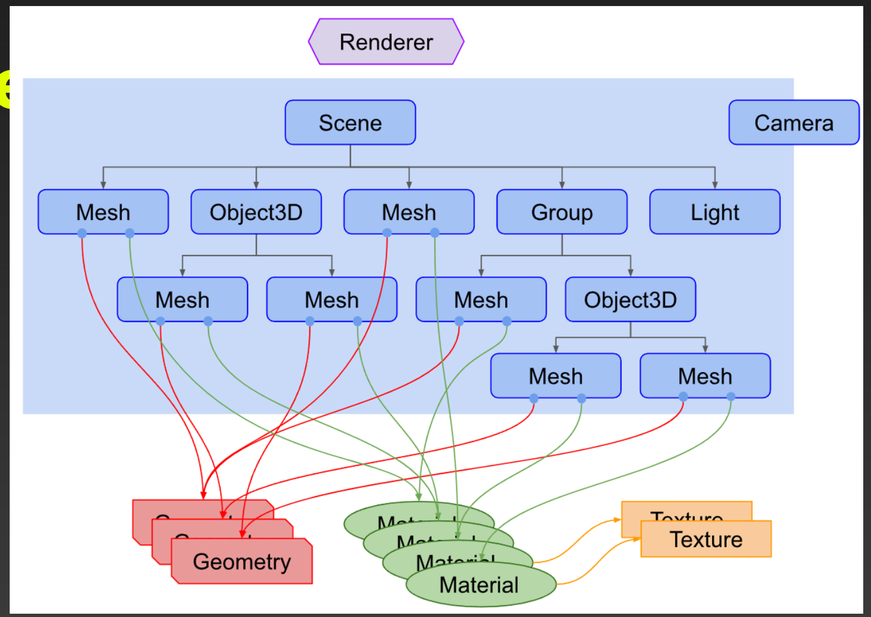
different material

different position and rotating angle

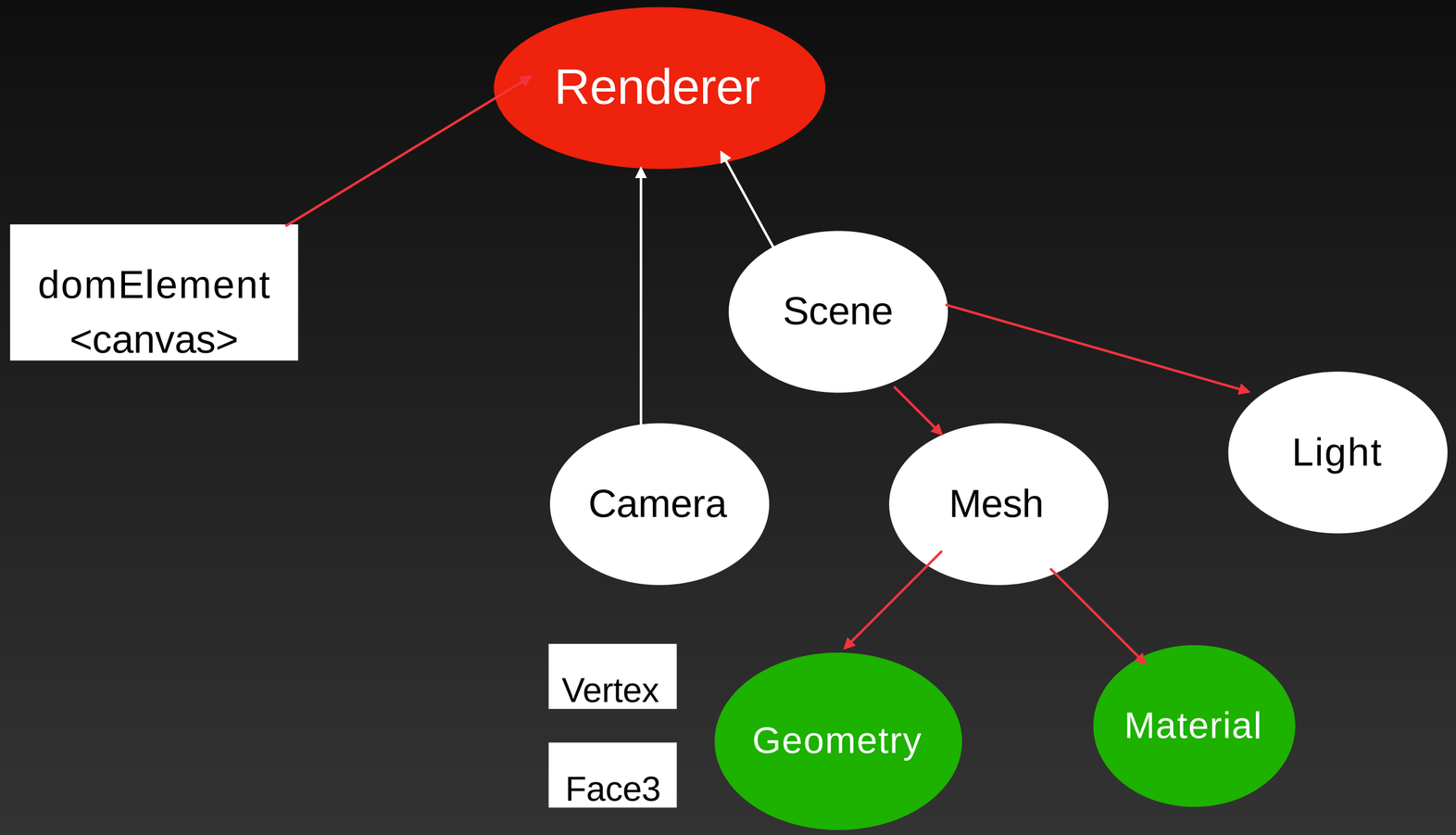


Scene Graph

- A three.js scene graph is made up of objects of type THREE.Object3D
- Cameras, lights, and visible objects are all represented by subclasse



Renderer & Scene Graph




```
renderer.render( scene, camera );
```

Transformation

- An Object3D, obj, has an associated transformation
  - obj.scale,
  - obj.rotation,
  - obj.position
  - obj.matrix —-The local transform matrix.
  - obj.matrixWorld global transformation matrix
  - obj.modelViewMatrix

- camera.position.z = 20;
- obj.scale.y = 0.5;

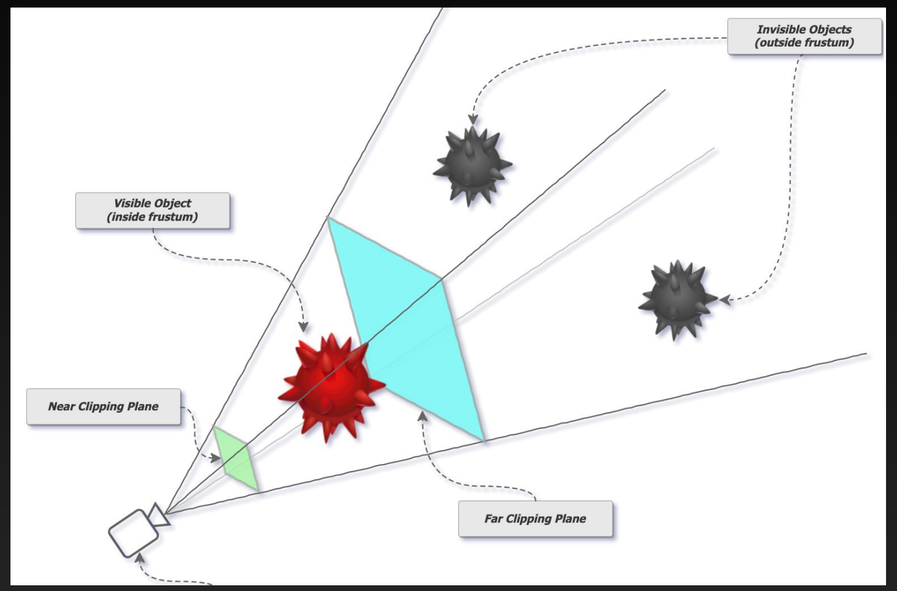
Scene

- `scene = new THREE.Scene();`
- `var node = new THREE.Object3D();`
- `var group = new THREE.Group();`
- `scene.add(group);`
- `group.add(node);`

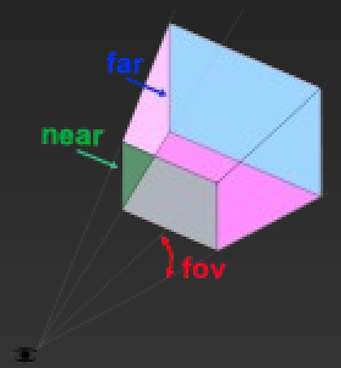
Camera

PerspectiveCamera(fov : Number, aspect : Number, near : Number, far : Number)
fov — Camera frustum vertical field of view. aspect — Camera frustum aspect ratio. near — Camera frustum near plane. far — Camera frustum far plane.

define the camera's viewing frustum.




```
var camera = new THREE.PerspectiveCamera( 45,
canvas.clientWidth/canvas.clientHeight,
1, 1000 );
renderer.add( camera );
```

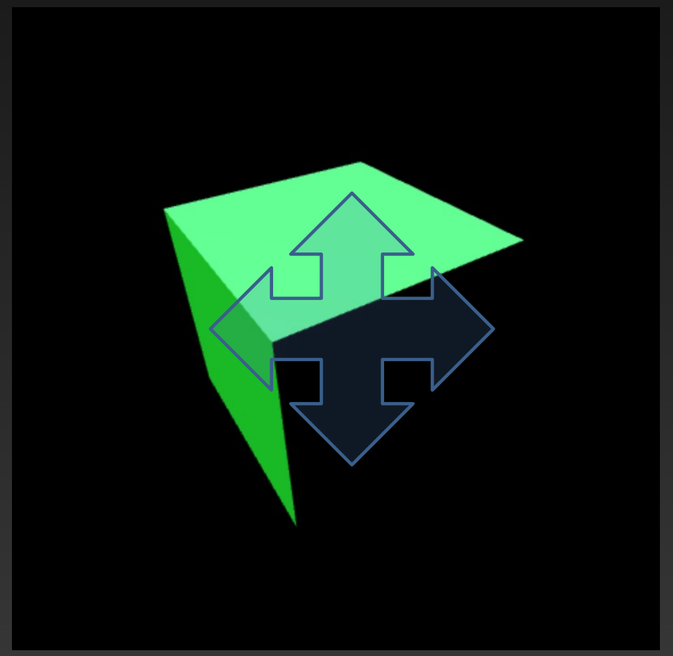


https://discoverthreejs.com/book/1-first-steps/2-lights-color-action/

Orbit control

Orbit controls allow the camera to orbit around a target.

```
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera,renderer.domElement);
camera.position.z = 2;
controls.update();
```



<script https://cdn.jsdelivr.net/npm/three-orbitcontrols@2.102.1/OrbitControls.js>

Responsive Design 響應式設計 (全螢幕)

```
<style>
html, body {
   margin: 0;
   height: 100%;
}
#c {
   width: 100%;
   height: 100%;
   display: block;
}
</style>
```

Webpage is displaying well on different sized displays from desktops to tablets to phones.

html and body height to 100% makes them fill the window.

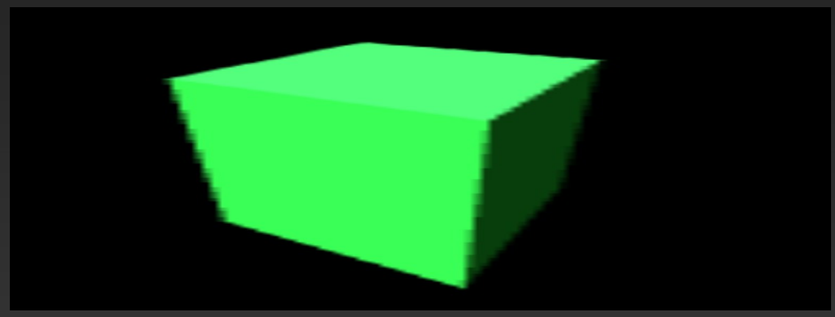
Set its display mode to block. A canvas's default display mode is inline. It can add whitespace to what is displayed.

Responsive Design 響應式設計 (全螢幕)

```
function render(time) {
  time *= 0.001;
 …
camera.aspect =
canvas.clientWidth /
canvas.clientHeight;
camera.updateProjectionMatrix();

  …
```

Fix the stretchy problem. We need to set the aspect of the camera. We can do that by looking at the canvas's clientWidth and clientHeight properties.



Responsive Design 響應式設計 (全螢幕)

```
function
resizeRendererToDisplaySize(renderer) {
   const needResize =
   (canvas.width !== canvas.clientWidth) ||
   (canvas.height !== canvas.clientHeight);

   if (needResize) {
   renderer.setSize(canvas.clientWidth,
   canvas.clientHeight, false);
   }
   return needResize;
}
```

Canvas elements have 2 sizes.
One size is the size the canvas is displayed on the page. The other size is the number of pixels in the canvas itself.
A canvas's internal size set by calling renderer.setSize.
Set false, then canvases used by three is not different than other elements.

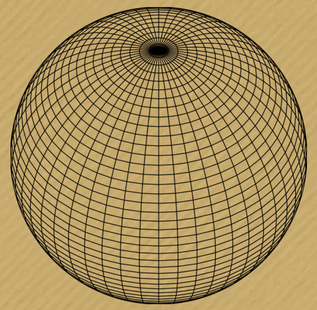
Responsive Design 響應式設計 (全螢幕)

```
function render(time) {
  time *= 0.001;
 …

if (resizeRendererToDisplaySize(renderer))
{
camera.aspect = canvas.clientWidth /
canvas.clientHeight;
camera.updateProjectionMatrix();
}

  ...
```

Mesh



Geometry



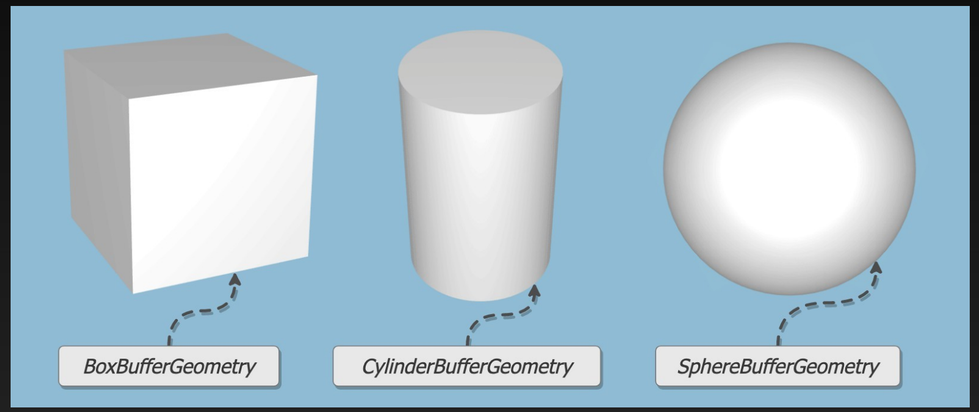
Materia
l

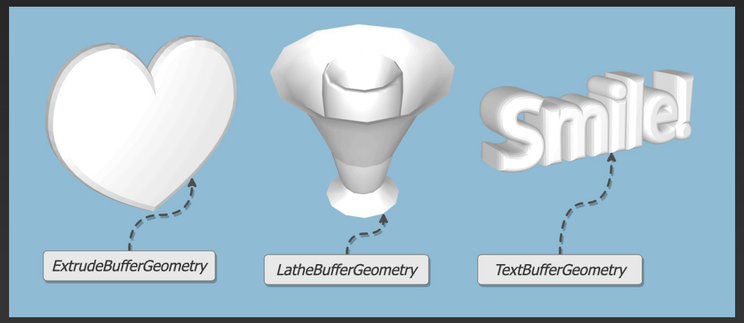


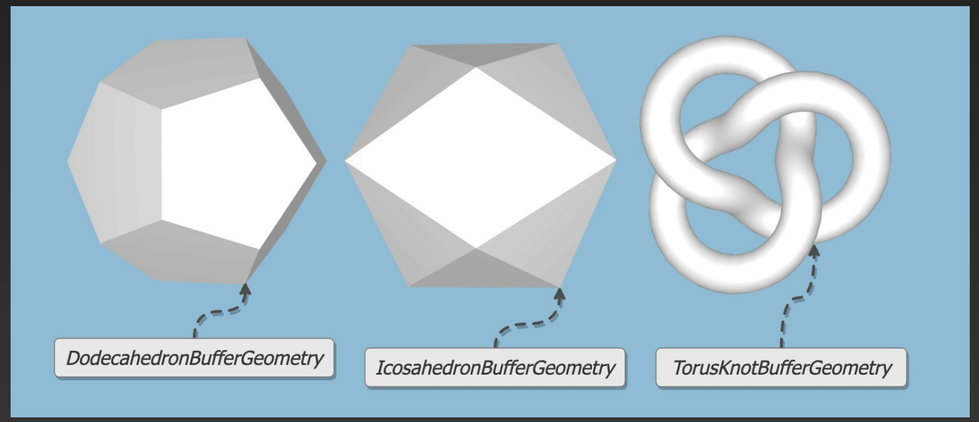
https://www.deviantart.com/phospheneart/art/Material-Spheres-481302617

Geometry

BoxGeometry
ConeGeometry
SphereGeometry
Cylinder**Geometry**
TorusGeometry
PlaneGeometry







COMMON OBJECT PROPERTIES

- Geometry
  - sphere.geometry
    - sphere.geometry.vertices
    - sphere.geometry.faces
- Transformaton
  - sphere.position *// contains x, y and z*
  - sphere.rotation *// same*
  - sphere.scale *// ... same*

BoxGeometry

BoxGeometry(width : Float, height : Float, depth : Float, widthSegments : Integer, heightSegments : Integer, depthSegments : Integer)

```
var geometry = new THREE.BoxGeometry( 1, 1, 1 );
var material = new THREE.MeshBasicMaterial( {color: 0x00ff00} );
var cube = new THREE.Mesh( geometry, material );
scene.add( cube );
```

SphereGeometry

SphereGeometry(radius : Float, widthSegments : Integer, heightSegments : Integer, phiStart : Float, phiLength : Float, thetaStart : Float, thetaLength : Float)

```
var sgeometry = new THREE.SphereGeometry( 1, 32, 32 );
var smaterial = new THREE.MeshBasicMaterial( {color: 0xffff00} );
var spmesh = new THREE.Mesh( sgeometry, smaterial );
scene.add( spmesh );
```

Other Geometry

```
const geometry = new THREE.ConeGeometry(
radius, height, radialSegments );

const geometry = new THREE.CylinderGeometry(
radiusTop, radiusBottom, height, radialSegments );

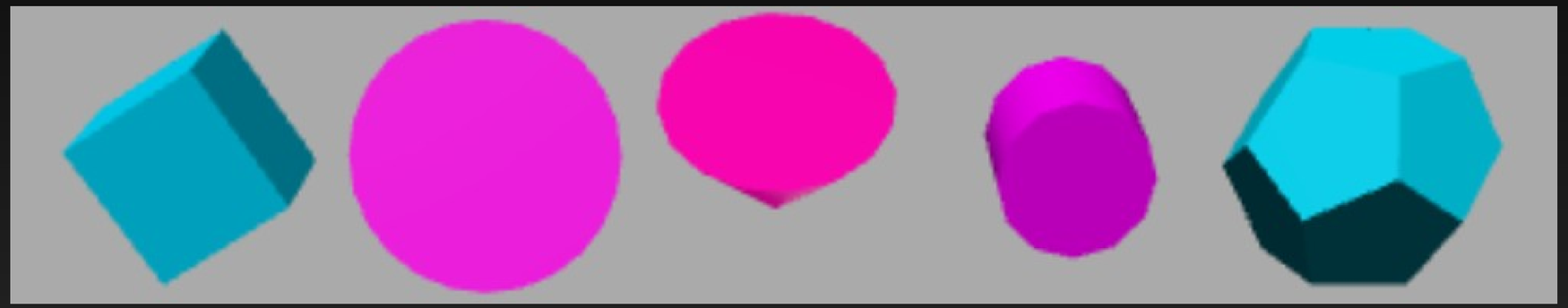
const geometry = new THREE.IcosahedronGeometry( radius );

const geometry = new THREE.TorusGeometry(
radius, tubeRadius, radialSegments, tubularSegments );

const geometry = new THREE.TorusKnotGeometry(
radius, tubeRadius, tubularSegments, radialSegments, p, q );
```

Lab

Try to draw the following meshes with self rotation animation

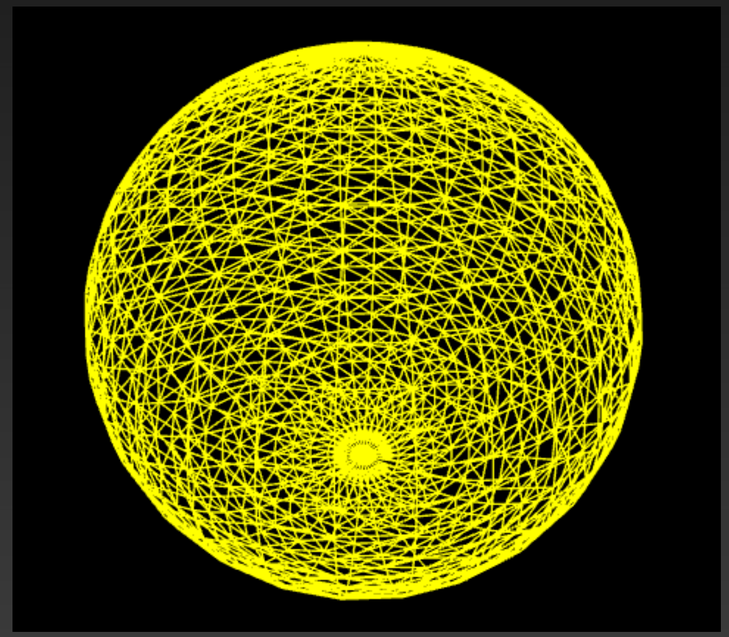


WireframeGeometry

```
const geometry = new THREE.SphereGeometry( 100, 100, 100 );

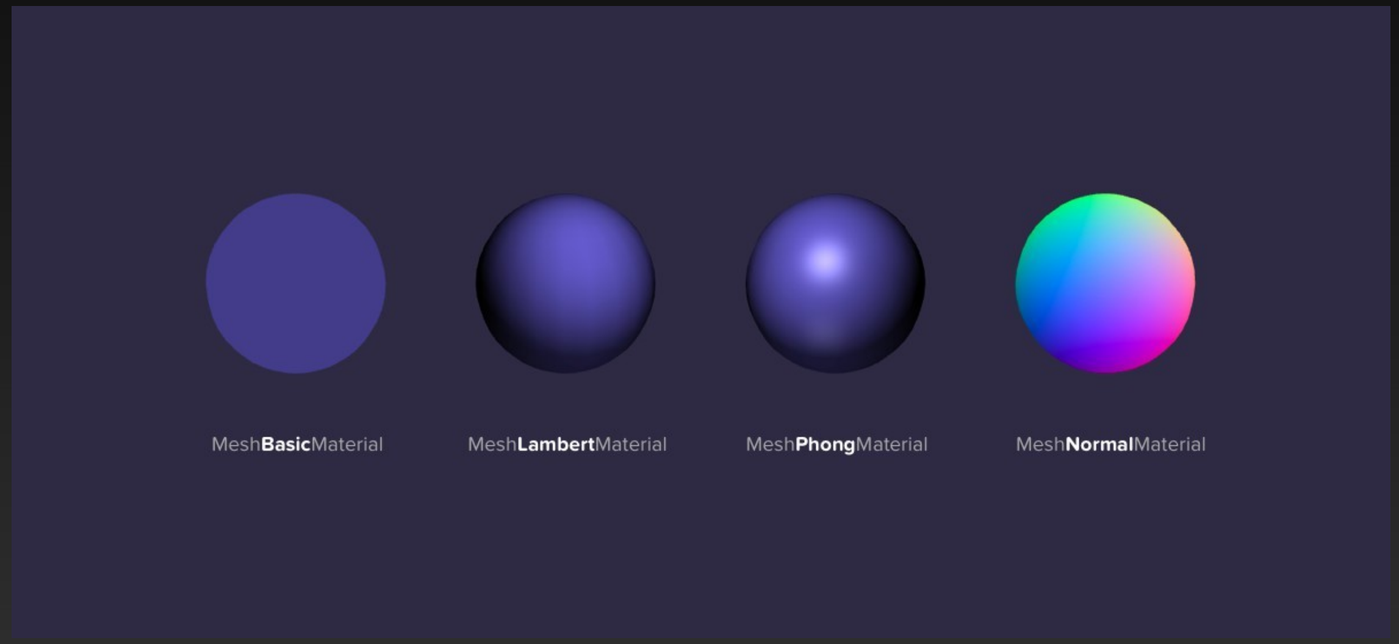
const material = new THREE.MeshBasicMaterial({ color: 0xffff00,
wireframe: true});

const mesh = new THREE.Mesh( geometry, material );
```



Material

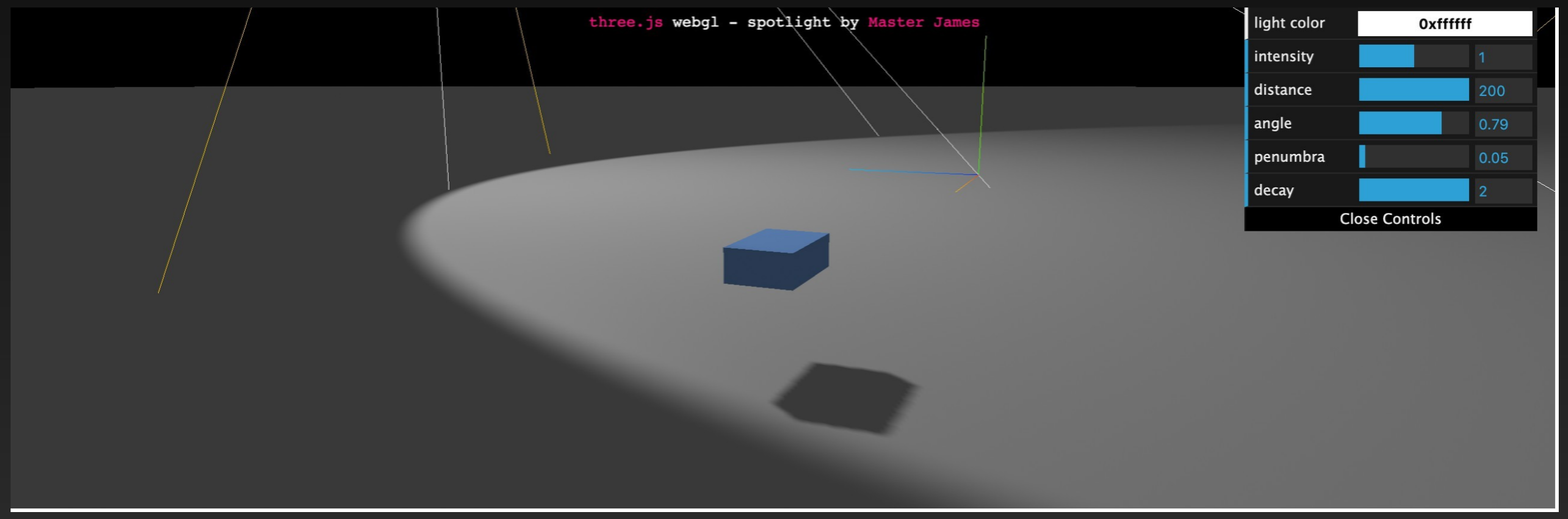
- MeshBasic**Material**
- MeshLambert**Material**
- MeshNormal**Material**
- MeshPhong**Material**
-
- RawShader**Material**
- Shader**Material**
- Shadow**Material**
- Sprite**Material**



Lights

- Ambient**Light**
- Directional**Light**
- Hemisphere**Light**

- Point**Light**
- RectArea**Light**
- Spot**Light**



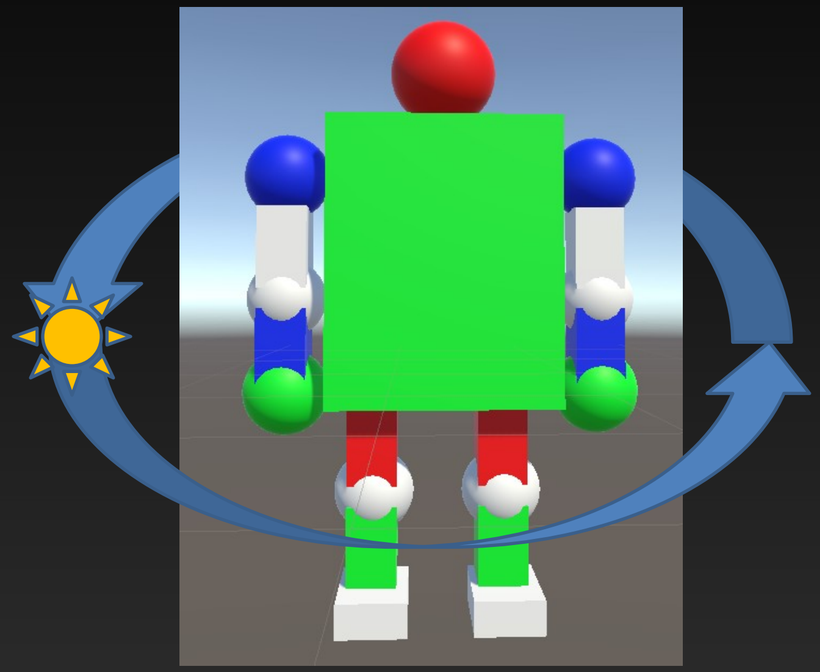
Homework_2

1. 請使用 geometry 相關指令 (如 BoxGeometry, ConeGeometry, SphereGeometry, CylinderGeometry, TorusGeometry 等) ，組合為一個簡單的三維機器人模型

2. 請不要讀入外部模型檔案 (如 obj 檔) 來繪製機器人模型，而是使用項目 1 的指令來表現

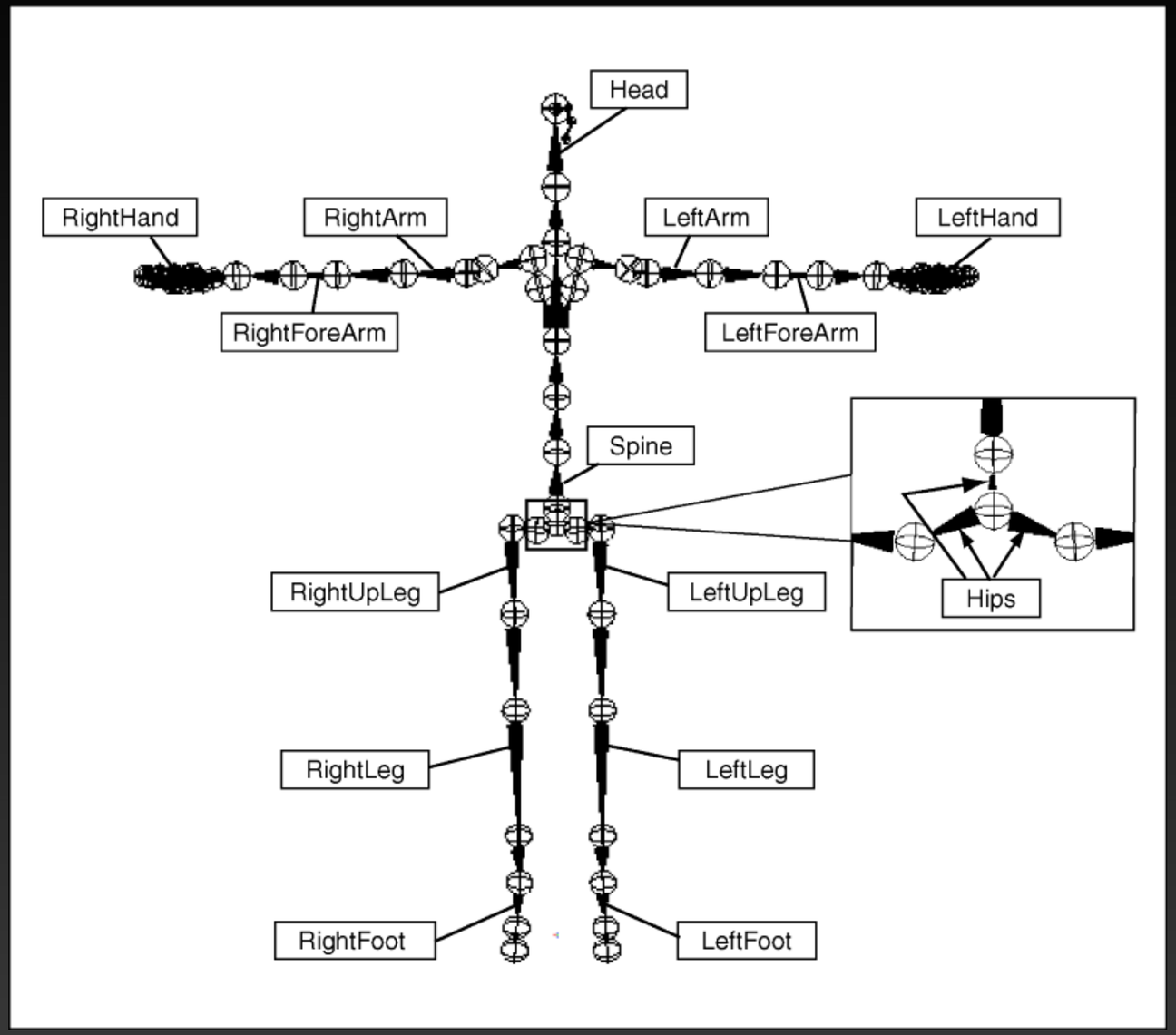
3. 建立一個點光源 (PointLight) 圍繞著機器人旋轉的動畫

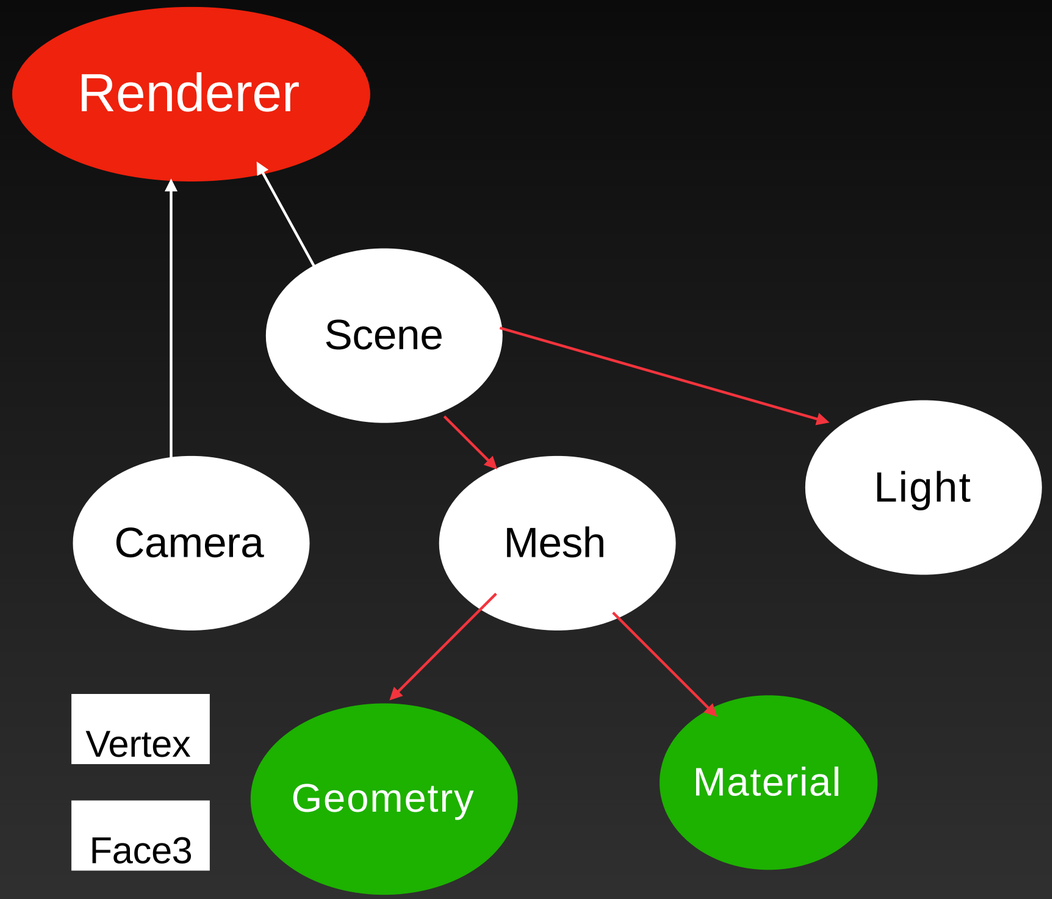
Optional ：請利用移動、旋轉、縮放指令，創作機器人動畫

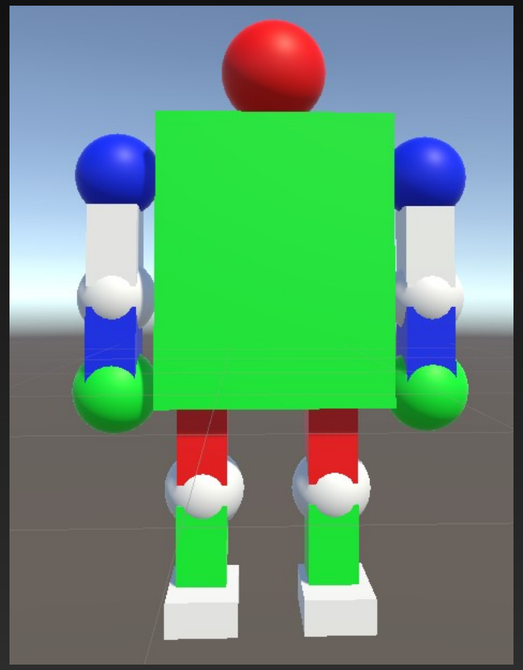


*** 11/13 demo Homework_1 與 Homework_2**

*** 11/13 demo 前請將 Homework_1 與 Homework_2 相關檔案壓縮後上傳雲端學院**


Head
RightHand
RightArm
LeftArm
LeftHand
RightForeArm
LeftForeArm
Spine
RightUpLeg
LeftUpLeg
Hips
RightLeg
LeftLeg
RightFoot
LeftFoot


Renderer
Scene
Camera
Mesh
Light
Vertex
Face3
Geometry
Material



RightShoulder

Head

LeftShoulder

RightArm

Body

LeftArm

RightJointArm

LeftJointArm

RightForeArm

LeftForeArm

RightHand

LeftHand

腳節點設計類似手，主要是讓大腿與小腿有適當旋轉的節點